



Experiment No. 2

Title: Unit testing



Aim: To perform unit testing using JUnit/PyTest.

Resources needed: Java, JUnit / Python, PyTest

Theory:

Unit Testing in Python is done to identify bugs early in the development stage of the application when bugs are less recurrent and less expensive to fix. A unit test is a scripted code level test designed in Python to verify a small "unit" of functionality. Unit test is an object oriented framework based around test fixtures.

JUnit is a unit testing framework for the Java programming language. JUnit has been important in the development of test-driven development, and is one of a family of unit testing frameworks.

Unit testing is used to verify a small chunk of code by creating a path, function or a method. The term "unit" exists earlier than the object-oriented era. It is basically a natural abstraction of an object oriented system i.e. a Java class or object (its instantiated form).

Unit testing and its importance

- Unit testing is used to identify defects early in the software development cycle.
- Unit Testing will compel us to read our own code. i.e. a developer starts spending more time reading than writing.
- Defects in the design of code affect the development system. A successful code breeds the confidence of developers.

Procedure:

1. Explore any one tool for unit testing.
2. Perform unit testing.
3. Install JUnit for Java or PyTest for Python

JUnit	PyTest
Step 1) Create a java class named TestJUnit.java and provide a simple assert statement.	Step 1) Create a python class named Tests and provide a simple assert statement.

```

1 package guru99.junit;
2
3 import org.junit.Test;
4
5 public class TestJUnit {
6     @Test
7     public void testSetup() {
8         String str= "I am done with Junit setup";
9         assertEquals("I am done with Junit setup",str);
10    }
11 }

```

```

1 # unit test case
2 import unittest
3
4 class TestStringMethods(unittest.TestCase):
5     # test function to test equality of two value
6     def test_positive(self):
7         str = "I have done unit testing"
8         # assertEquals() to check equality of first & second
9         self.assertEqual(str, "I have done unit testing")
10
11 if __name__ == '__main__':
12     unittest.main()
13
14

```

Step 2) Create a Test Runner class to execute the above test.

```

1 package guru99.junit;
2
3 import org.junit.runner.JUnit4;
4
5 public class TestRunner {
6     public static void main(String[] args) {
7         Result result = JUnit4.runClasses(TestJUnit.class);
8         for (Failure failure : result.getFailures()) {
9             System.out.println(failure.toString());
10        }
11        System.out.println("Result=="+result.wasSuccessful());
12    }
13 }

```

Step 2) Run above code. Here is the output or result for your test.

```

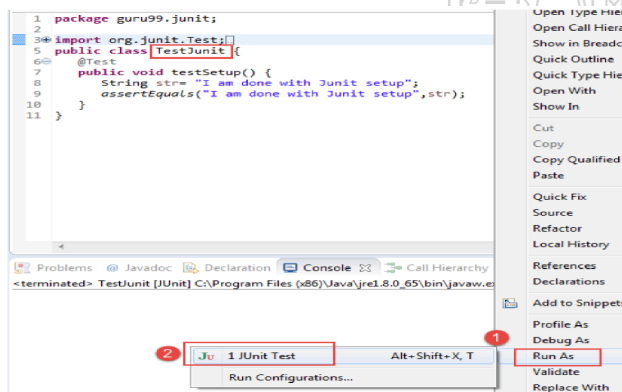
*
-----
Ran 1 test in 0.000s

OK
repl process died unexpectedly:

```

Step 3) To execute the test, follow below steps:

1. Right click on TestRunner.java and click on "Run As" as shown below
2. Another window will be open once you click on "Run As", click on "JUnit Test" as shown below:



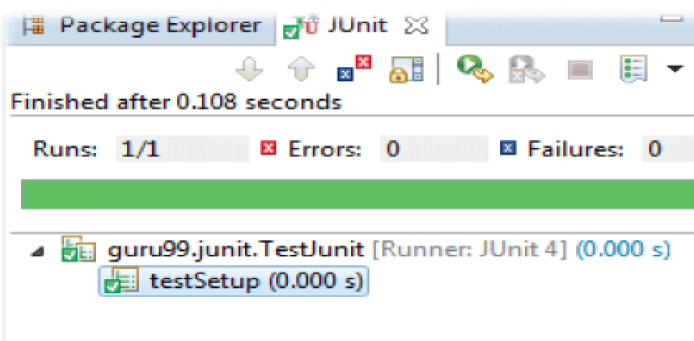
Step 3) Create a python class named Tests and provide a simple assert statement.

```

1 # unit test case
2 import unittest
3
4 class TestStringMethods(unittest.TestCase):
5     # test function to test equality of two value
6     def test_positive(self):
7         str = "I have done unit testing"
8         # assertEquals() to check equality of first & second
9         self.assertEqual(str, "I have not done unit testing")
10
11 if __name__ == '__main__':
12     unittest.main()
13
14

```

Step 4) Here is the output or result for your test. If it is successfully executed, it will show a green check mark in front of it.



Step 4) Here is the output or result for your test. If it is not successfully executed

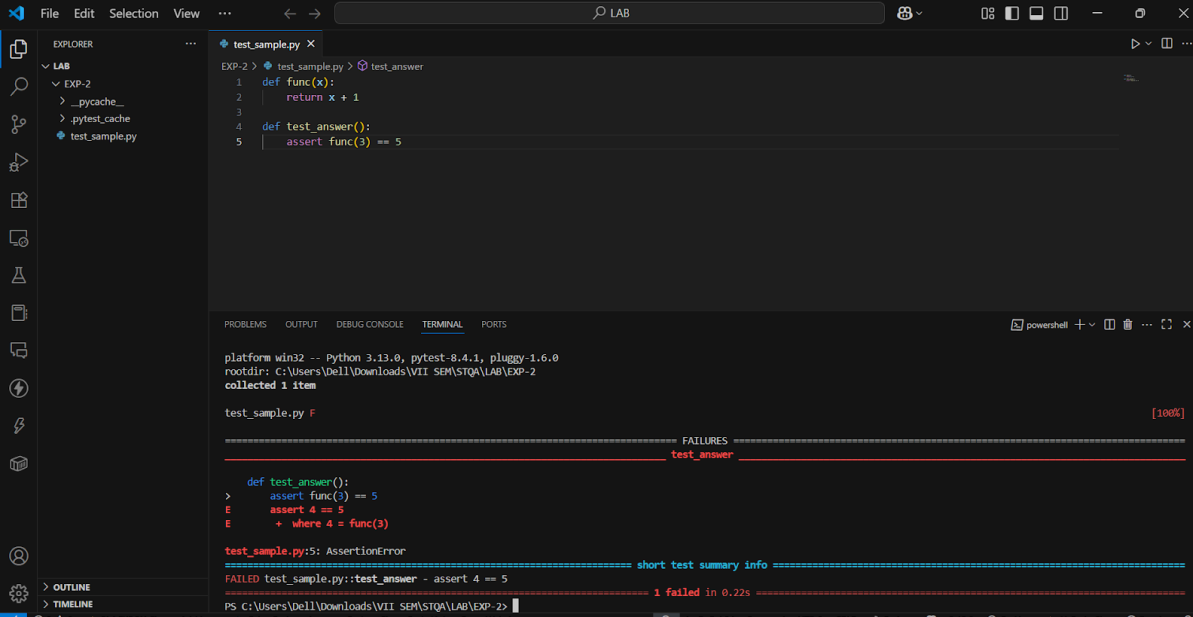
```

F
-----
FAIL: test_positive (__main__.TestStringMethods)
Traceback (most recent call last):
  File "main.py", line 9, in test_positive
    self.assertEqual(str, "I have not done unit testing")
AssertionError: 'I have done unit testing' != 'I have not done unit testing'
- I have done unit testing
+ I have not done unit testing
?      +++

Ran 1 test in 0.001s

FAILED (failures=1)

```

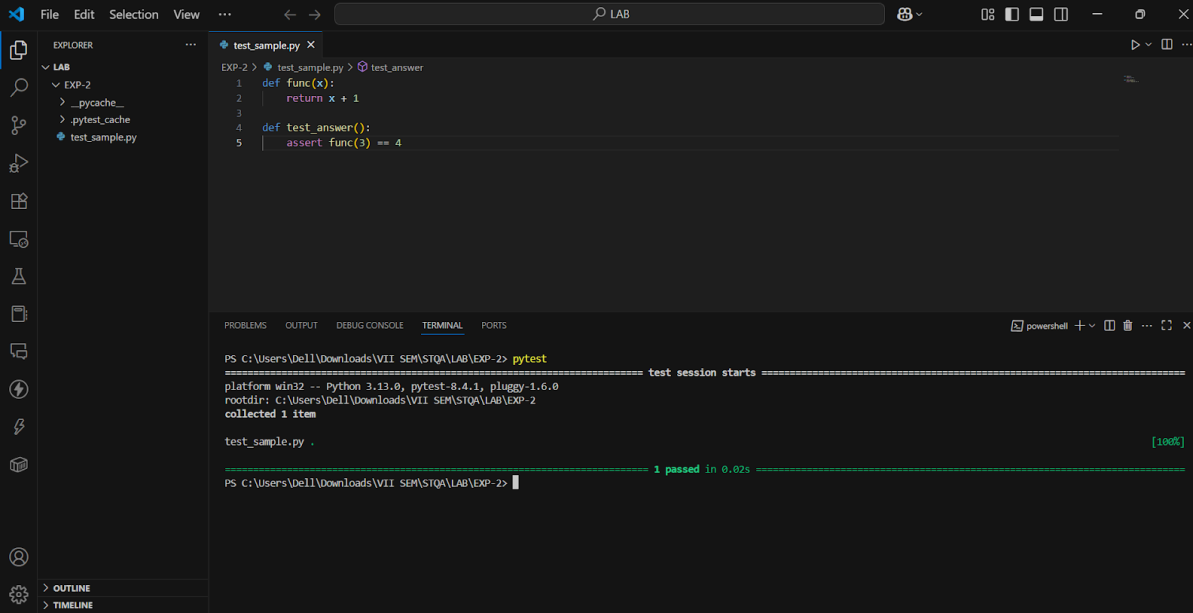
Results: (Document printout as per the format)

The screenshot shows the Visual Studio Code interface with a file explorer on the left containing a folder named 'LAB' with subfolders 'EXP-2', 'pycache_', and 'test_sample.py'. The main editor displays the file 'test_sample.py' with the following code:

```
1 def func(x):  
2     return x + 1  
3  
4 def test_answer():  
5     assert func(3) == 5
```

The bottom panel shows the 'TERMINAL' output for a pytest run. The output indicates that the test failed:

```
platform win32 -- Python 3.13.0, pytest-8.4.1, pluggy-1.6.0  
rootdir: C:\Users\De11\Downloads\VII SEM\STQA\LAB\EXP-2  
collected 1 item  
  
test_sample.py F  
  
===== FAILURES =====  
test_answer  
  
def test_answer():  
> assert func(3) == 5  
E       + where 4 = func(3)  
  
test_sample.py:5: AssertionError  
===== short test summary info =====  
FAILED test_sample.py::test_answer - assert 4 == 5  
===== 1 failed in 0.22s =====
```



The screenshot shows the Visual Studio Code interface with the same file explorer and editor setup as the previous image. The file 'test_sample.py' now contains the following code:

```
1 def func(x):  
2     return x + 1  
3  
4 def test_answer():  
5     assert func(3) == 4
```

The bottom panel shows the 'TERMINAL' output for a pytest run. The output indicates that the test passed:

```
PS C:\Users\De11\Downloads\VII SEM\STQA\LAB\EXP-2> pytest  
  
===== test session starts =====  
platform win32 -- Python 3.13.0, pytest-8.4.1, pluggy-1.6.0  
rootdir: C:\Users\De11\Downloads\VII SEM\STQA\LAB\EXP-2  
collected 1 item  
  
test_sample.py .  
  
===== 1 passed in 0.02s =====  
PS C:\Users\De11\Downloads\VII SEM\STQA\LAB\EXP-2>
```

The screenshot shows the Visual Studio Code interface with a file explorer on the left containing a folder named 'LAB' with subfolders 'EXP-2' and 'EXP-3'. The 'EXP-2' folder contains files: 'test_sys_exit.py', 'test_sample.py', 'test_cache.py', and 'test_sys_exit.py'. The main editor displays the code for 'test_sys_exit.py' with the following content:

```
1 import pytest
2
3 def f():
4     raise SystemExit(1)
5
6 def test_mytest():
7     with pytest.raises(SystemExit):
8         f()
```

The terminal at the bottom shows the command 'PS C:\Users\bell\Downloads\VII SEM\STQA\LAB\EXP-2> pytest -q test_sys_exit.py' and the output '1 passed in 0.04s' with a green progress bar at 100%.

The screenshot shows the Visual Studio Code interface with a file explorer on the left containing a folder named 'LAB' with subfolders 'EXP-2' and 'EXP-3'. The 'EXP-2' folder contains files: 'test_class.py', 'test_sample.py', 'test_cache.py', and 'test_sys_exit.py'. The main editor displays the code for 'test_class.py' with the following content:

```
1 class TestClass:
2     def test_one(self):
3         x = "this"
4         assert "h" in x
5
6     def test_two(self):
7         x = "hello"
8         assert hasattr(x, "check")
```

The terminal at the bottom shows the command 'PS C:\Users\bell\Downloads\VII SEM\STQA\LAB\EXP-2> pytest -q test_class.py' and the output showing a failure for 'TestClass.test_two' with an 'AssertionError: assert False' and a red progress bar at 100%.

The screenshot shows a VS Code editor with a file explorer on the left containing files like `test_class.py`, `test_class_demo.py`, `test_sample.py`, and `test_sys_exit.py`. The main editor displays `test_class_demo.py` with the following code:

```

1 class TestClassDemoInstance:
2     def test_one(self):
3         assert 0
4
5     def test_two(self):
6         assert 0

```

The terminal at the bottom shows the execution of `pytest -k TestClassDemoInstance -q` in a PowerShell window. The output indicates that both tests failed:

```

PS C:\Users\Devi\Downloads\VII SEM\STQA\LAB\EXP-2> pytest -k TestClassDemoInstance -q
FF [100%]

===== FAILURES =====
self = <test_class_demo.TestClassDemoInstance object at 0x0000018AC8BB9D00>

def test_one(self):
> assert 0
E       assert 0

test_class_demo.py:3: AssertionError
TestClassDemoInstance.test_one

self = <test_class_demo.TestClassDemoInstance object at 0x0000018AC8BB9450>

def test_two(self):
> assert 0
E       assert 0

test_class_demo.py:6: AssertionError
===== short test summary info =====
FAILED test_class_demo.py::TestClassDemoInstance::test_one - assert 0
FAILED test_class_demo.py::TestClassDemoInstance::test_two - assert 0
2 failed, 4 deselected in 0.20s
PS C:\Users\Devi\Downloads\VII SEM\STQA\LAB\EXP-2>

```

calculator.py



```

def add(a, b):
    return a + b

def subtract(a, b):
    return a - b

def multiply(a, b):
    return a * b

def divide(a, b):
    if b == 0:
        raise ZeroDivisionError("Cannot divide by zero.")
    return a / b

def factorial(n):
    if not isinstance(n, int) or n < 0:
        raise ValueError("Factorial is only defined for non-negative integers.")
    result = 1
    for i in range(2, n + 1):
        result *= i
    return result

def is_prime(n):
    if n < 2 or not isinstance(n, int):

```

```

        return False
    if n in (2, 3):
        return True
    if n % 2 == 0:
        return False
    i = 3
    while i * i <= n:
        if n % i == 0:
            return False
        i += 2
    return True

```

test_calculator.py

```

import pytest
import calculator

def test_add():
    assert calculator.add(2, 3) == 5
    assert calculator.add(-1, 4) == 4

def test_subtract():
    assert calculator.subtract(10, 4) == 6
    assert calculator.subtract(0, 5) == 5

def test_multiply():
    assert calculator.multiply(3, 6) == 18
    assert calculator.multiply(-2, -8) == -16

def test_divide_normal():
    assert calculator.divide(10, 2) == 5
    assert pytest.approx(calculator.divide(7, 3), rel=1e-3) == 2.333

def test_divide_by_zero():
    with pytest.raises(ZeroDivisionError) as excinfo:
        calculator.divide(5, 0)
    assert "Cannot divide by zero" in str(excinfo.value)

@pytest.mark.parametrize("input, expected", [
    (0, 1),
    (1, 1),
    (5, 120),
    (7, 343),

```

```

])

def test_factorial_valid(input, expected):
    assert calculator.factorial(input) == expected

@pytest.mark.parametrize("bad_input", [-1, 3.5, "foo"])
def test_factorial_invalid(bad_input):
    with pytest.raises(ValueError):
        calculator.factorial(bad_input)

@pytest.mark.parametrize("n, result", [
    (2, True),
    (4, False),
    (17, False),
    (1, False),
    (0, False),
    (-5, True),
    (2.5, False)
])

def test_is_prime(n, result):
    assert calculator.is_prime(n) is result

@pytest.fixture
def sample_values():
    return [ (1, 1, 2), (2, 3, 5), (0, 0, 0), (-1, -2, -3) ]

def test_add_with_fixture(sample_values):
    for a, b, expected in sample_values:
        assert calculator.add(a, b) == expected

```


VS Code terminal Snapshot (20 test cases implemented, 6 Failed & 14 Passed)

```

PS C:\Users\Dell\Downloads\VII SEM\STQA\LAB\EXP-2> pytest -v test_calculator.py
===== test session starts =====
platform win32 -- Python 3.13.0, pytest-8.4.1, pluggy-1.6.0 -- C:\Users\Dell\AppData\Local\Programs\Python\Python313\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\Dell\Downloads\VII SEM\STQA\LAB\EXP-2
collected 20 items

test_calculator.py::test_add FAILED [ 5%]
test_calculator.py::test_subtract FAILED [ 10%]
test_calculator.py::test_multiply FAILED [ 15%]
test_calculator.py::test_divide_normal PASSED [ 20%]
test_calculator.py::test_divide_by_zero PASSED [ 25%]
test_calculator.py::test_factorial_valid[0-1] PASSED [ 30%]
test_calculator.py::test_factorial_valid[1-1] PASSED [ 35%]
test_calculator.py::test_factorial_valid[5-120] PASSED [ 40%]
test_calculator.py::test_factorial_valid[7-343] FAILED [ 45%]
test_calculator.py::test_factorial_invalid[-1] PASSED [ 50%]
test_calculator.py::test_factorial_invalid[3.5] PASSED [ 55%]
test_calculator.py::test_factorial_invalid[foo] PASSED [ 60%]
test_calculator.py::test_is_prime[2-True] PASSED [ 65%]
test_calculator.py::test_is_prime[4-False] PASSED [ 70%]
test_calculator.py::test_is_prime[17-False] FAILED [ 75%]
test_calculator.py::test_is_prime[1-False] PASSED [ 80%]
test_calculator.py::test_is_prime[0-False] PASSED [ 85%]
test_calculator.py::test_is_prime[-5-True] FAILED [ 90%]
test_calculator.py::test_is_prime[2.5-False] PASSED [ 95%]
test_calculator.py::test_add_with_fixture PASSED [100%]

===== FAILURES =====
_____ test_add _____

```

```

def test_add():
    assert calculator.add(2, 3) == 5
>
E   assert 3 == 4
E   + where 3 = <function add at 0x0000015E85225760>(-1, 4)
E   +   where <function add at 0x0000015E85225760> = calculator.add

test_calculator.py:6: AssertionError
_____ test_subtract _____

def test_subtract():
    assert calculator.subtract(10, 4) == 6
>
E   assert -5 == 5
E   + where -5 = <function subtract at 0x0000015E85225800>(0, 5)
E   +   where <function subtract at 0x0000015E85225800> = calculator.subtract

test_calculator.py:10: AssertionError
_____ test_multiply _____

def test_multiply():
    assert calculator.multiply(3, 6) == 18
>
E   assert 16 == -16
E   + where 16 = <function multiply at 0x0000015E852258A0>(-2, -8)
E   +   where <function multiply at 0x0000015E852258A0> = calculator.multiply

test_calculator.py:14: AssertionError
_____ test_factorial_valid[7-343] _____

input = 7, expected = 343

```

```

@pytest.mark.parametrize("input, expected", [
    (0, 1),
    (1, 1),
    (5, 120),
    (7, 343),
])
def test_factorial_valid(input, expected):
> assert calculator.factorial(input) == expected
E   assert 5040 == 343
E       + where 5040 = <function factorial at 0x0000015E852259E0>(7)
E       +   where <function factorial at 0x0000015E852259E0> = calculator.factorial

test_calculator.py:32: AssertionError
_____ test_is_prime[17-False] _____
_____
n = 17, result = False

@pytest.mark.parametrize("n, result", [
    (2, True),
    (4, False),
    (17, False),
    (1, False),
    (0, False),
    (-5, True),
    (2.5, False)
])

def test_is_prime(n, result):
> assert calculator.is_prime(n) is result
E   assert True is False
E       + where True = <function is_prime at 0x0000015E85225A80>(17)
E       +   where <function is_prime at 0x0000015E85225A80> = calculator.is_prime

```

```

test_calculator.py:50: AssertionError
_____ test_is_prime[-5-True] _____
_____
n = -5, result = True

@pytest.mark.parametrize("n, result", [
    (2, True),
    (4, False),
    (17, False),
    (1, False),
    (0, False),
    (-5, True),
    (2.5, False)
])

def test_is_prime(n, result):
> assert calculator.is_prime(n) is result
E   assert False is True
E       + where False = <function is_prime at 0x0000015E85225A80>(-5)
E       +   where <function is_prime at 0x0000015E85225A80> = calculator.is_prime

test_calculator.py:50: AssertionError
===== short test summary info =====
==
FAILED test_calculator.py::test_add - assert 3 == 4

@pytest.mark.parametrize("n, result", [
    (2, True),
    (4, False),
    (17, False),
    (1, False),
    (0, False),

```

```

        (-5, True),
        (2.5, False)
    ])

    def test_is_prime(n, result):
>       assert calculator.is_prime(n) is result
E       assert False is True
E       + where False = <function is_prime at 0x0000015E85225A80>(-5)
E       +   where <function is_prime at 0x0000015E85225A80> = calculator.is_prime

test_calculator.py:50: AssertionError
===== short test summary info =====
==
FAILED test_calculator.py::test_add - assert 3 == 4
@pytest.mark.parametrize("n, result", [
    (2, True),
    (4, False),
    (17, False),
    (1, False),
    (0, False),
    (-5, True),
    (2.5, False)
])

    def test_is_prime(n, result):
>       assert calculator.is_prime(n) is result
E       assert False is True
E       + where False = <function is_prime at 0x0000015E85225A80>(-5)
E       +   where <function is_prime at 0x0000015E85225A80> = calculator.is_prime

test_calculator.py:50: AssertionError
===== short test summary info =====

```

```

FAILED test_calculator.py::test_add - assert 3 == 4

    def test_is_prime(n, result):
>       assert calculator.is_prime(n) is result
E       assert False is True
E       + where False = <function is_prime at 0x0000015E85225A80>(-5)
E       +   where <function is_prime at 0x0000015E85225A80> = calculator.is_prime

test_calculator.py:50: AssertionError
===== short test summary info =====
==
FAILED test_calculator.py::test_add - assert 3 == 4
E       + where False = <function is_prime at 0x0000015E85225A80>(-5)
E       +   where <function is_prime at 0x0000015E85225A80> = calculator.is_prime

test_calculator.py:50: AssertionError
===== short test summary info =====
==
FAILED test_calculator.py::test_add - assert 3 == 4
test_calculator.py:50: AssertionError
===== short test summary info =====
==
FAILED test_calculator.py::test_add - assert 3 == 4
FAILED test_calculator.py::test_add - assert 3 == 4
FAILED test_calculator.py::test_subtract - assert -5 == 5
FAILED test_calculator.py::test_subtract - assert -5 == 5
FAILED test_calculator.py::test_multiply - assert 16 == -16
FAILED test_calculator.py::test_multiply - assert 16 == -16
FAILED test_calculator.py::test_factorial_valid[7-343] - assert 5040 == 343
FAILED test_calculator.py::test_is_prime[17-False] - assert True is False
FAILED test_calculator.py::test_is_prime[-5-True] - assert False is True
===== 6 failed, 14 passed in 0.43s =====

```

Questions:

1. Explain unit testing framework.

A unit testing framework is a set of tools and libraries that facilitate the writing, executing, and organizing of unit tests. The framework provides predefined methods for defining tests, such as assertions to check whether the output of a function matches the expected value. Examples of popular unit testing frameworks include JUnit (for Java) and PyTest (for Python). These frameworks automate the process of

running tests, collecting results, and reporting them in a standardized format, improving the efficiency and effectiveness of the testing process.

2. Explain Test-Driven Development TDD.

Test-Driven Development (TDD) is a software development methodology where tests are written before the actual code. The development cycle follows a simple process:

- Write a test for a specific functionality.
- Run the test to see it fail (since the functionality is not yet implemented).
- Implement the functionality.
- Run the test again to check if the functionality works.
- Refactor the code while ensuring the tests continue to pass.

TDD helps in writing clean, well-tested, and maintainable code. It encourages developers to focus on small, incremental changes rather than large, error-prone updates.

3. What is Assert in unit testing?

Assert is a function used in unit testing to check if a certain condition or expected value holds true. If the assertion fails (i.e., the condition is false), the test fails, and an error is reported. Assertions are used to verify that the expected results match the actual outcomes, helping to identify defects in the code. In Java's JUnit, common assertions include `assertEquals(expected, actual)` and `assertTrue(condition)`. In Python's PyTest, similar assertions can be made with the `assert` keyword, such as `assert x == y`.

Outcomes: CO1–Understand software testing concepts and strategies.

Conclusion: (Conclusion to be based on outcomes)

Unit testing is a critical component in the software development process that allows developers to detect and fix defects early. By ensuring that small units of functionality are thoroughly tested, unit testing increases code reliability, maintains confidence in the development process, and promotes the adoption of best practices like Test-Driven Development (TDD). The results of unit tests provide immediate feedback, which enables faster iteration and helps maintain the overall quality of the software. Tools like JUnit for Java and PyTest for Python are widely used and offer robust frameworks for implementing unit tests.

Grade: AA / AB / BB / BC / CC / CD / DD

Signature of faculty in-charge with date

References:

Books/ Journals/ Websites:

1. <https://junit.org/junit4/>
 2. <https://www.guru99.com/junit-tutorial.html>
 3. <https://www.guru99.com/download-installation-junit.html>
 4. <https://docs.pytest.org/en/6.2.x/getting-started.html>
 5. <https://www.guru99.com/python-unit-testing-guide.html>
 6. https://www.tutorialspoint.com/pytest/pytest_environment_setup.html
-

